6.NVIC与EXTI

一、中断

1.1 中断的概念

在我们前面的学习中，已经了解到了GPIO相关用法。那么，假如我们要完成用一个按键去控制LED亮灭的程序，很容易想到这样的写法：

代码块

```
while(1){
    if(HAL_GPIO_ReadPin(KEY_PORT, KEY_PIN) == GPIO_PIN_RESET){
        HAL_GPIO_TogglePin(LED_PORT, LED_PIN);
    }
}
```

这么做虽然可以控制小灯亮灭，但如果想要让程序在小灯亮灭的同时完成其他效果，比如说控制呼吸灯效果，就会造成CPU被不停的"读取按键"这件事情所绑死，而无法执行。

那我们就会开始想，能不能让CPU该干嘛干嘛，只有当按键按下去的时候再进行处理？

这个想法的最终体现就是 **中断**。通俗来讲，中断就是打断CPU执行正常的程序，转而处理紧急程序，处理完后返回原来正常程序暂停的位置，并继续运行。

1.2 前置知识：函数指针与回调函数

在正式讲NVIC和EXTI之前，我们需要先补充一个C语言的知识点——**回调函数**。

要理解回调函数，得先从**函数指针**说起。

函数指针：指向函数的指针

我们已经学过指针——指针是一个变量，存的是某个数据的地址。比如 `int *p` 存的是一个 `int` 变量的地址。

那函数有没有地址？当然有。程序运行时，函数的代码也是存在内存里的，函数名本身就是这个函数的入口地址。既然有地址，就能用指针存起来——**存函数地址的指针，就叫函数指针。**

来看一个最简单的例子：

代码块

```c
#include <stdio.h>

// 定义一个普通函数
void say_hello(int num)
{
    printf("Hello! 你传入的数字是：%d\n", num);
}

int main()
{
    // 定义一个函数指针 p
    // 返回值是 void，参数是一个 int
    void (*p)(int);

    // 让 p 指向 say_hello 函数（函数名就是地址，不用加 &）
    p = say_hello;

    // 通过函数指针调用函数，两种写法都可以
    p(100);           // 写法一：直接当函数名用
    (*p)(200);        // 写法二：先解引用再调用

    return 0;
}
```

运行结果：

代码块

```
Hello! 你传入的数字是: 100
Hello! 你传入的数字是: 200
```

函数指针的定义语法稍微有点绕，记住这个格式就行：

代码块

```
返回值类型 (*指针名)(参数类型列表);
```

比如 `void (*p)(int)` 的意思是：定义一个指针 `p` ，它指向的函数返回值是 `void` ，接受一个 `int` 参数。

回调函数：被别人调用的函数

有了函数指针，回调函数就很好理解了。

回调函数就是一个通过函数指针调用的函数。 你把一个函数的地址（函数名）作为参数传给另一个函数，当这个函数在合适的时候用函数指针调用你传进去的那个函数，被调用的那个函数就是回调函数。

听起来有点绕，用一个生活中的例子就清楚了。

> 假设你点外卖：
>
> - 平台会把你的电话号码告诉小哥。这里的"电话号码"就相当于**函数指针**——它指向了"通知你"这个动作。
> - 小哥把外卖送到宿舍楼下（这是朋友自己的主任务）。
> - 完成后，小哥拨打电话通知你（小哥通过你留下的"函数指针"，调用了你提供的"回调函数"）。

> 整个过程中，你不需要在楼下干等，送到后小哥会自动通知你。这就是回调的核心思想：**把"做什么"和"什么时候做"分开——你负责写"做什么"，别人负责在合适的时候调用它。**

来看一个代码例子，体会一下回调的用法：

代码块

```c
#include <stdio.h>

// 回调函数1：打印元素
void print_element(int num)
{
    printf("%d ", num);
}

// 回调函数2：打印元素的平方
void print_square(int num)
{
    printf("%d ", num * num);
}

// 这个函数遍历数组，对每个元素调用回调函数 action
// action 就是一个函数指针，接受一个 int 参数，返回 void
void for_each(int *arr, int len, void (*action)(int))
{
    for (int i = 0; i < len; i++)
    {
        action(arr[i]);  // 调用回调函数
    }
    printf("\n");
}

int main()
{
    int arr[] = {1, 2, 3, 4, 5};
    int len = 5;

    printf("原数组: ");
    for_each(arr, len, print_element);    // 传入 print_element 作为回调

    printf("平方后: ");
    for_each(arr, len, print_square);     // 传入 print_square 作为回调

    return 0;
```

```
}
```

运行结果：

代码块

```
原数组：1 2 3 4 5
平方后：1 4 9 16 25
```

看到了吗？for_each函数的遍历逻辑是固定的，但**每个元素具体做什么**，由传入的回调函数决定。同一个for_each，传print_element就打印原值，传print_square就打印平方。这就是回调的威力：**把流程和具体动作解耦，同一个流程可以搭配不同的动作。**

弱函数：可以被覆盖的默认实现

在嵌入式开发中，回调函数经常和一个叫做**弱函数**的东西搭配使用。

弱函数用__attribute__((weak))修饰，特点是：**它提供了一个默认实现，但如果用户自己写了一个同名的函数（不加weak），用户写的那个就会覆盖默认实现。**

来看一个例子：

代码块

```c
#include <stdio.h>

// 弱函数：默认实现
__attribute__((weak)) void on_button_pressed(void)
{
    printf("【默认】按键按下了，但用户没有写处理逻辑\n");
}

int main()
{
```

```
    printf("模拟按键按下...\n");
    on_button_pressed();  // 调用回调
    return 0;
}
```

如果用户什么都不写，运行结果是：

代码块

```
模拟按键按下...
【默认】按键按下了，但用户没有写处理逻辑
```

但如果用户在另一个文件里写了一个同名函数（不加weak）：

代码块

```
// 用户自己写的，覆盖弱函数
void on_button_pressed(void)
{
    printf("【用户】按键按下了，我要点亮LED! \n");
}
```

运行结果就变成了：

代码块

```
模拟按键按下...
【用户】按键按下了，我要点亮LED!
```

> **一句话总结弱函数：** 库的作者给你留了一个"默认是空的"函数，你需要的时候就自己写一个同名的把它覆盖掉，不需要就不管它，程序也能正常跑。这是HAL库最核心的设计思想之一。

好了，前置知识补充完毕。接下来我们正式进入NVIC的学习。

二、NVIC——中断管家

2.1 前言

在我们目前所使用的STM32系列芯片当中，设置了专门用来处理中断的控制器——**NVIC**，即嵌套向量中断控制器（Nested Vectored Interrupt Controller）。NVIC通过中断优先级管理、嵌套中断处理以及中断向量的跳转来管理和协调处理器的中断请求，是STM32中处理异步事件的重要机制。

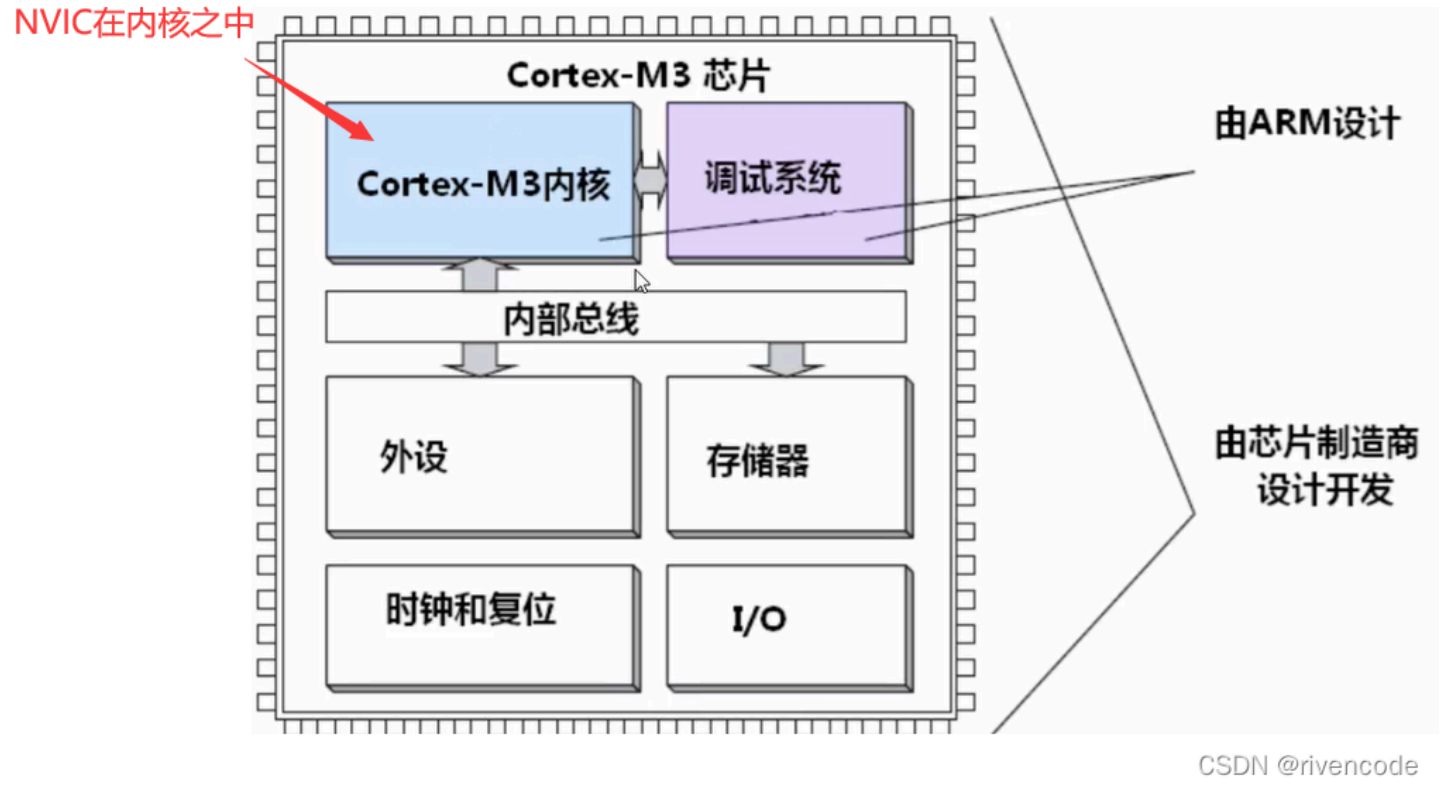
说白了，NVIC就是专门帮CPU管中断的管家。

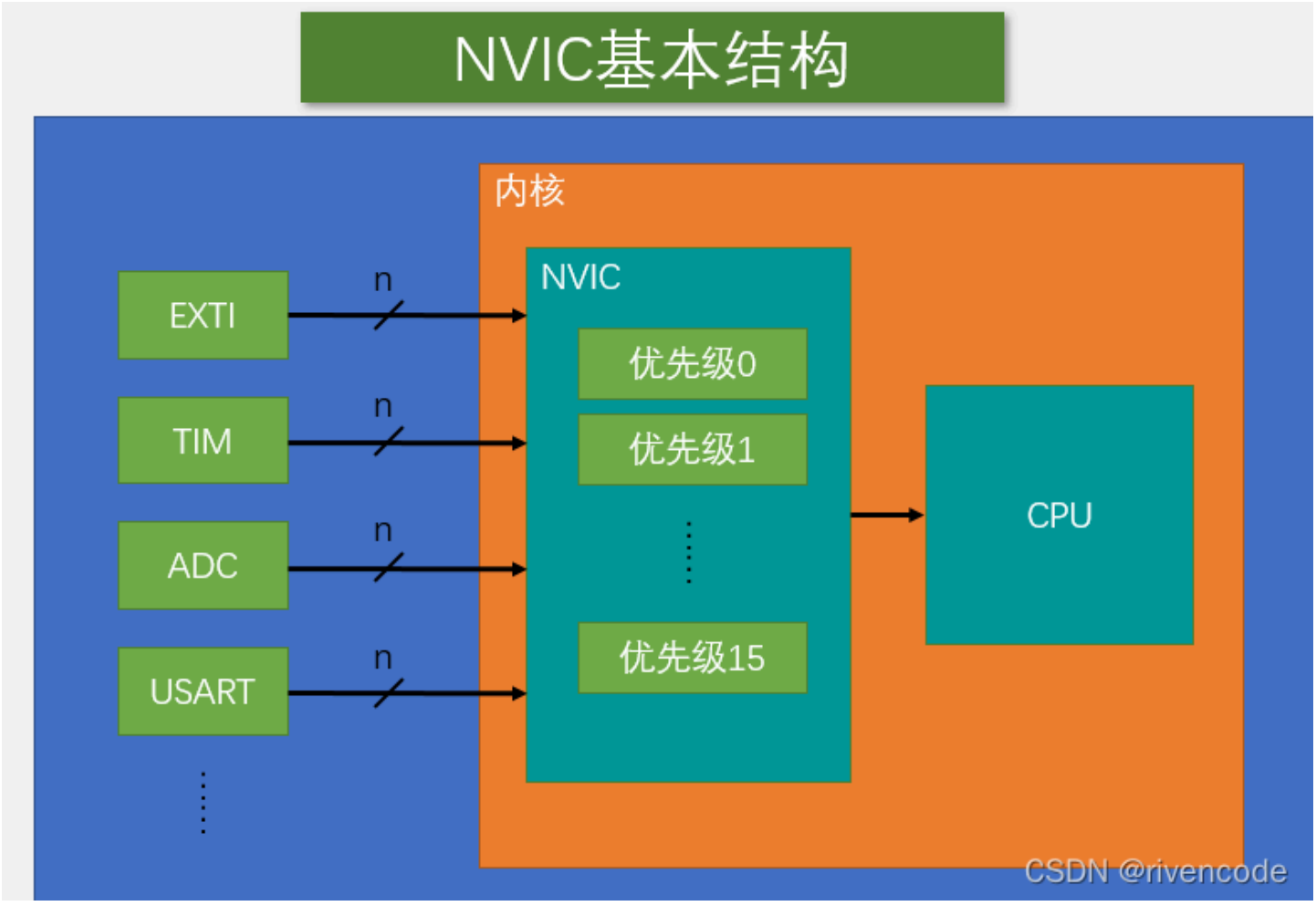
2.2 NVIC的硬件原理

既然叫"中断管家"，那这个管家具体干些什么呢？我们来拆解一下。

NVIC是Cortex-M内核的一部分，属于**内核外设**——它和CPU是一体的，不是外挂在总线上的片上外设。所有中断（包括GPIO外部中断、定时器中断、串口中断等等）都要先经过NVIC，才能到达CPU。

> 这里区分一下**内核外设**和**片上外设**：它们都在芯片里面，但内核外设是在CPU内核里面（比如NVIC、SysTick），片上外设是在内核之外、通过总线和CPU相连（比如GPIO、EXTI、USART、TIM）。NVIC是我们遇到的第一个内核外设。





管家的工作可以分成三步：

第一步：门卫——中断使能

每个中断都有一个独立的开关。开关打开，这个中断的请求才能被NVIC接收；开关关着，不管外部怎么喊，NVIC都直接忽略，CPU根本不知道有这件事。

> 就像管家先确认"今天见不见客"，不见的话，门口喊破喉咙也没用。

这个开关在硬件上由**中断使能寄存器（ISER）** 和 **中断失能寄存器（ICER**控制，后面讲寄存器时会细说。这里你只需要知道：**NVIC的第一道关卡是使能，没使能的中断进不来。**

第二步：调度员——优先级裁决

假设多个中断同时使能了，而且同时来了请求——比如按键按下的同时，串口也收到了数据——CPU先处理谁？

这就是NVIC的核心工作：**比较所有待处理中断的优先级，选出最高的那个，把它送到CPU。**

优先级分两级：

- **抢占优先级**：决定能不能插队
- **响应优先级**：抢占优先级一样时，决定谁先排队

> 就像管家同时收到好几个客人的拜访请求，按客人的重要程度排个顺序，最重要的先见。

第三步：管家的核心能力——中断嵌套

这是NVIC最厉害的地方，也是它名字里"**Nested（嵌套）**"的由来。

NVIC会时刻盯着：**现在CPU正在处理哪个中断？它的抢占优先级是多少？**

如果此时来了一个新中断，NVIC会比较：

- 新中断的抢占优先级 **更高** → 让CPU暂停当前中断，先处理这个新的，处理完再回来继续。这就是**中断嵌套**。
- 新中断的抢占优先级 **更低或相等** → 排队等着，等当前中断处理完再说。

> 就像你正在跟一个普通客人谈话，突然老板来了电话——你得暂停谈话先接老板的电话，接完再回来继续聊。老板的电话就是高抢占优先级的中断。

注意：嵌套是NVIC硬件自动完成的，不需要你在代码里写判断逻辑。 你只需要配置好优先级，剩下的NVIC自己搞定。

中断向量表——管家和CPU之间的通讯录

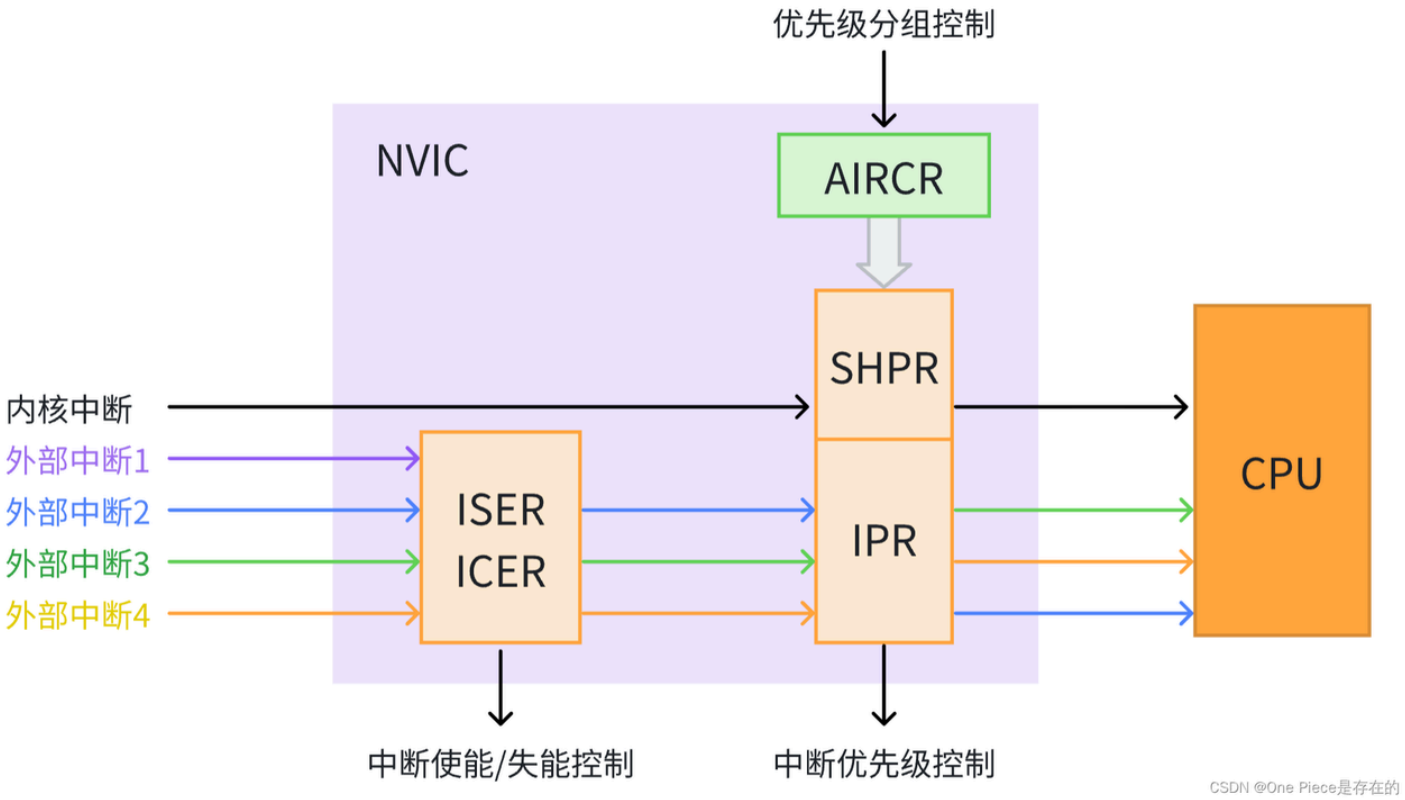
NVIC裁决完之后，并不直接告诉CPU"去执行哪个函数"，它只告诉CPU一个编号："该处理第x号中断了。"

CPU拿到编号后，去一张叫做**中断向量表**的表里查地址——这张表里存着每一个中断对应的服务函数的入口地址，就像一个通讯录：编号对应函数地址。CPU查到地址后，跳过去执行，执行完再回到原来被打断的地方继续。

> 管家只说"见3号客人"，CPU自己翻通讯录找到3号客人在哪个房间，然后过去。

NVIC完整工作流程

把上面三步串起来，NVIC处理一个中断的完整流程是：



当外部被触发时，首先进入到ISER、ICER寄存器，来判断是否开启对应的中断，中断开启后，进入到IPR寄存器，进行中断优先级判断，其中IPR寄存器受到AIRCR寄存器控制，最后NVIC会确定其优先级并根据优先级决定传入CPU的次序。
接下来，进入CPU的事件根据优先级抢占CPU使用权，然后CPU会查询中断向量表,以查找到相应的中断服务程序地址。最后,CPU会将控制转移到该地址,以执行相应的中断服务。

2.3 NVIC相关寄存器

NVIC相关的寄存器有很多，这里主要讲比较重要的几个。

名称	位数	个数	作用
中断使能寄存器 ISER	32	8	每一位控制一个中断（打开）
中断失能寄存器 ICER	32	8	每一位控制一个中断（关闭）
应用程序中断及复位控制寄存器 AIRCR	32	1	位[10:8]控制中断优先级分组
中断优先级寄存器 IPR	8	240	

			8位对应一个中断， STM32只使用高4位

ISER 中断使能寄存器组

《STM32F10xxx Cortex-M3编程手册》对ISER寄存器的描述如下：

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SETENA[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **SETENA[31:0]**: Interrupt set-enable bits.

Write:

0: No effect

1: Enable interrupt

Read:

0: Interrupt disabled

1: Interrupt enabled.

CSDN @Daemon.Chen

> Cortex-M3内核支持 256 个中断，这里用 8 个 32 位寄存器（即8个ISER）来控制，每个位控制一个中断。但是STM32 的可屏蔽中断最多只有 68 个（互联型），所以就是三个（ISER[0~2]），总共可以表示 96 个中断。而 STM32 只用了其中的前 68 位。ISER[0]的 bit0~31 分别对应中断0~31；ISER[1]的 bit0~32 对应中断 32~63；ISER[2]的 bit0~3 对应中断 64~67；这样总共 68 个中断就分别对应上了。 若使能某个中断，必须设置相应的 ISER 位为 1

ICER 中断储能寄存器组

《STM32F10xxx Cortex-M3编程手册》对ICER寄存器的描述如下：

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The ICER0-ICER2 registers disable interrupts, and show which interrupts are enabled.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRENA[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRENA[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:0 **CLRENA[31:0]**: Interrupt clear-enable bits.

Write:

0: No effect

1: Disable interrupt

Read:

0: Interrupt disabled

1: Interrupt enabled.

CSDN @Daemon.Chen

> 该寄存器组用来清除某个中断的使能，**若失能某个中断，必须设置相应的ICER位为1**

AIRCR 应用程序中断及复位控制寄存器

《STM32F10xxx Cortex-M3编程手册》对AIRCR寄存器的描述如下：

Address offset: 0x0C

Reset value: 0xFA05 0000

Required privilege: Privileged

The AIRCR provides priority grouping control for the exception model, endian status for data accesses, and reset control of the system.

To write to this register, you must write 0x5FA to the VECTKEY field, otherwise the processor ignores the write.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
VECTKEYSTAT[15:0](read)/ VECTKEY[15:0](write)															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ENDIANESS	Reserved				PRIGROUP			Reserved					SYSRESETREQ	VECTCLRACTIVE	VECTRESET
r					rw	rw	rw						w	w	w

Bits 31:16 **VECTKEYSTAT[15:0]/ VECTKEY[15:0]** Register key
Reads as 0xFA05
On writes, write 0x5FA to VECTKEY, otherwise the write is ignored.

Bit 15 ENDIANESS Data endianness bit
Reads as 0.
0: Little-endian

Bits 14:11 Reserved, must be kept cleared

Bits 10:8 **PRIGROUP[2:0]**: Interrupt priority grouping field
This field determines the split of group priority from subpriority, see *Binary point on page 135*.

Bits 7:3 Reserved, must be kept cleared

Bit 2 **SYSRESETREQ** System reset request
This is intended to force a large system reset of all major components except for debug.
This bit reads as 0.
0: No system reset request
1: Asserts a signal to the outer system that requests a reset.

Bit 1 VECTCLRACTIVE
Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is unpredictable.

Bit 0 VECTRESET
Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is unpredictable.

CSDN @Daemon.Chen

> STM32将中断分为5个组，组0—4，该分组由AIRCR的[10:8]位决定，最终影响到IPR[7:4]位分配情况。具体见下面的中断优先级的章节。

IPR 中断优先级寄存器

《STM32F10xxx Cortex-M3编程手册》对IPR寄存器的描述如下：

Address offset: 0x00- 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The IPR0-IPR16 registers provide a 4-bit priority field for each interrupt. These registers are byte-accessible. Each register holds four priority fields, that map to four elements in the CMSIS interrupt priority array IP[0] to IP[67], as shown in *Figure 19*.

Figure 19. NVIC_IPRx register mapping

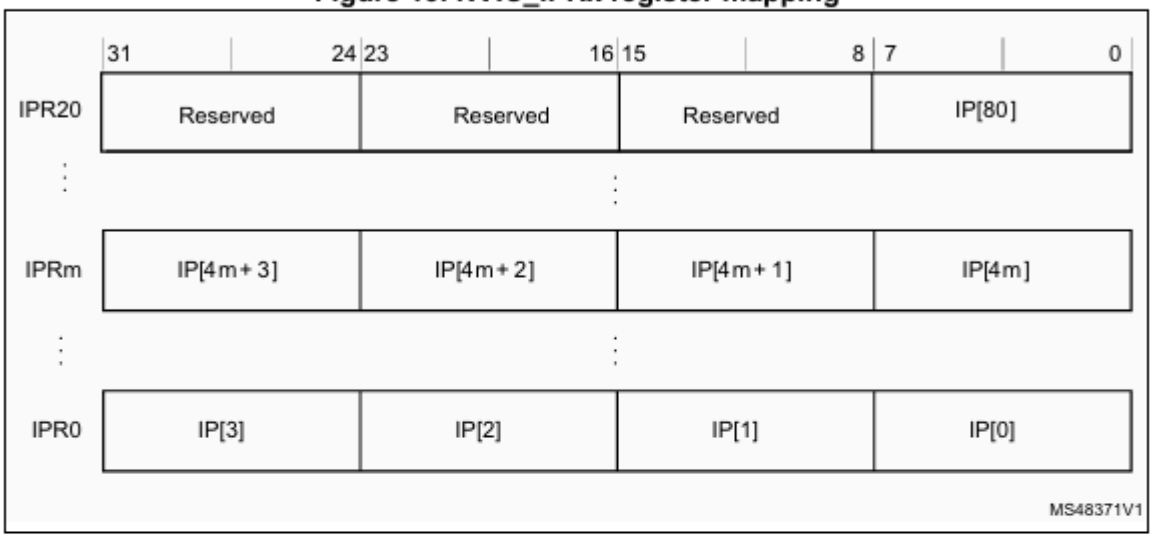


Table 42. IPR bit assignments

Bits	Name	Function
[31:24]	Priority, byte offset 3	Each priority field holds a priority value, 0-255. The lower the value, the greater the priority of the corresponding interrupt. The processor implements only bits[7:4] of each field, bits[3:0] read as zero and ignore writes.
[23:16]	Priority, byte offset 2	
[15:8]	Priority, byte offset 1	
[7:0]	Priority, byte offset 0	

See *The CMSIS mapping of the Cortex®-M3 NVIC registers on page 119* for more information about the IP[0] to IP[67] interrupt priority array, that provides the software view of the interrupt priorities.

Find the IPR number and byte offset for interrupt *N* as follows:

- The corresponding IPR number, *M*, is given by *M* = *N* DIV 4
- The byte offset of the required Priority field in this register is *N* MOD 4, where:
  - byte offset 0 refers to register bits[7:0]
  - byte offset 1 refers to register bits[15:8]
  - byte offset 2 refers to register bits[23:16]
  - byte offset 3 refers to register bits[31:24].

CSDN @Daemon.Chen

> IPR是控制中断优先级的一个非常重要的寄存器，STM32 的中断分组与这个寄存器组密切相关。IPR 寄存器组由 240 个 8bit 的寄存器组成，每个可屏蔽中断占用 8bit，这样总共可以表示 240 个可屏蔽中断。而 STM32 只用到了其中的 68 个。IP[67]~IP[0]分别对应中断 67~0。而每个可屏蔽中断占用的 8bit 并没有全部使用，而是 只用了高 4 位。这 4 位，又分为抢占优先级和子优先级。抢占优先级在前，子优先级在后。而这两个优先级各占几个位又要根据 SCB->AIRCR 中的中断分组设置来决定。

如果有感兴趣的同学可以看一下这些内容进行学习：

《STM32F10xxx Cortex-M3编程手册》

cm4 NVIC完全解析

2.4 中断优先级

在单片机执行主程序时，突然遇到了两个及两个以上的突发事件，单片机将先运行哪一部分程序呢？这便需要用中断优先级来处理。

中断优先级即给不同的中断划分不同的响应等级，如果多个中断同时发生，则STM32优先处理高优先级的中断。

中断优先级主要被分为两类：**抢占优先级**与**响应优先级**。

- **抢占优先级**是抢占式的（数值越小优先级越高，0 > 1 > 2 > 3……），即优先级高的中断到来时，即使优先级较低的中断正在执行，也要停下来先执行优先级较高的中断。

- **响应优先级**是非抢占式的，即当响应优先级比较低的中断在执行，如果抢占优先级相同，响应优先级较高的中断到来，那么要等这个响应优先级比较低的中断执行完，再执行后到来的中断。

当抢占优先级相同、响应优先级不同的两个中断同时需要被执行的时候，先执行响应优先级较高的中断。

2.5 优先级分组

那么NVIC是怎样将优先级进行分类的呢？

为了在有限的寄存器位数中实现更加丰富的中断优先级，NVIC使用了中断分组机制。STM32将中断进行分组，一共五组，即0~4。在实际应用中，我们通常根据具体需求选择合适的分组方式。分组后，再将优先级划分为抢占优先级与响应优先级。

优先级分组	AIRCR[10:8]	IPRx bit[7:4]分配	分配结果

0	111	None : [7:4]	0位抢占优先级，4位响应优先级
1	110	[7] : [6:4]	1位抢占优先级，3位响应优先级
2	101	[7:6] : [5:4]	2位抢占优先级，2位响应优先级
3	100	[7:5] : [4]	3位抢占优先级，1位响应优先级
4	011	[7:4] : None	4位抢占优先级，0位响应优先级

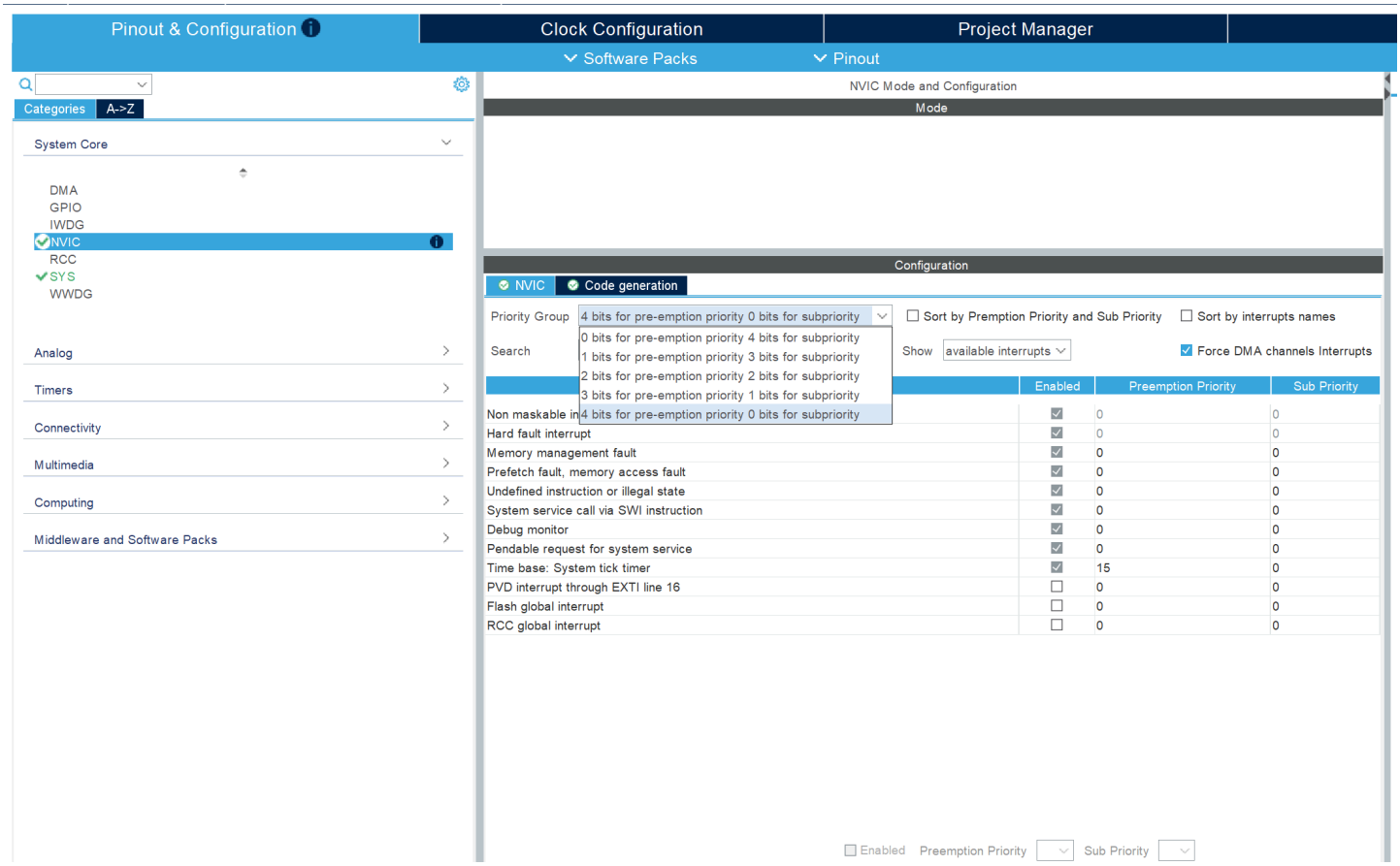
有4个比特用于优先级分组，当4比特都是响应优先级的时候（也就是2的4次方），响应优先级便有0~15共16个优先级，其他的同理，有几位就有2的几次方。比如优先级分组2中，分配结果是有2位抢占优先级、2位响应优先级，那么就代表有2的2次方即4个抢占优先级，有2的2次方即4个响应优先级。

下表便是全部情况：

优先级分组	抢占优先级	响应优先级
0	0位，取值为0	4位，取值为0-15
1	1位，取值为0-1	3位，取值为0-7
2	2位，取值为0-3	2位，取值为0-3
3	3位，取值为0-7	1位，取值为0-1
4	4位，取值为0-15	0位，取值为0

因此，抢占优先级和响应优先级的数量均可以通过NVIC中AIRCR寄存器的PRIGROUP[8:10]位进行配置，从而规定了两种优先级对NVIC_IPRx[7:4]的划分，根据划分决定两种优先级的数量。

明白了中断优先级的分类原理，有利于我们在实际应用中根据具体需求选择合适的分组方式。接下来我们来实际感悟一下吧，打开CubeMX，找到System Core下面的NVIC，在Priority Group的下拉框中，我们就可以自行配置不同的分组方式了。
如图所示：



2.6 中断嵌套

通过对抢占优先级与响应优先级的比较，我们了解到当高抢占优先级中断发生时，系统会暂停当前执行的中断（无论其优先级高低），转而处理高优先级中断。这种特性我们将其称为**中断嵌套**。中断嵌套是NVIC的核心功能之一。

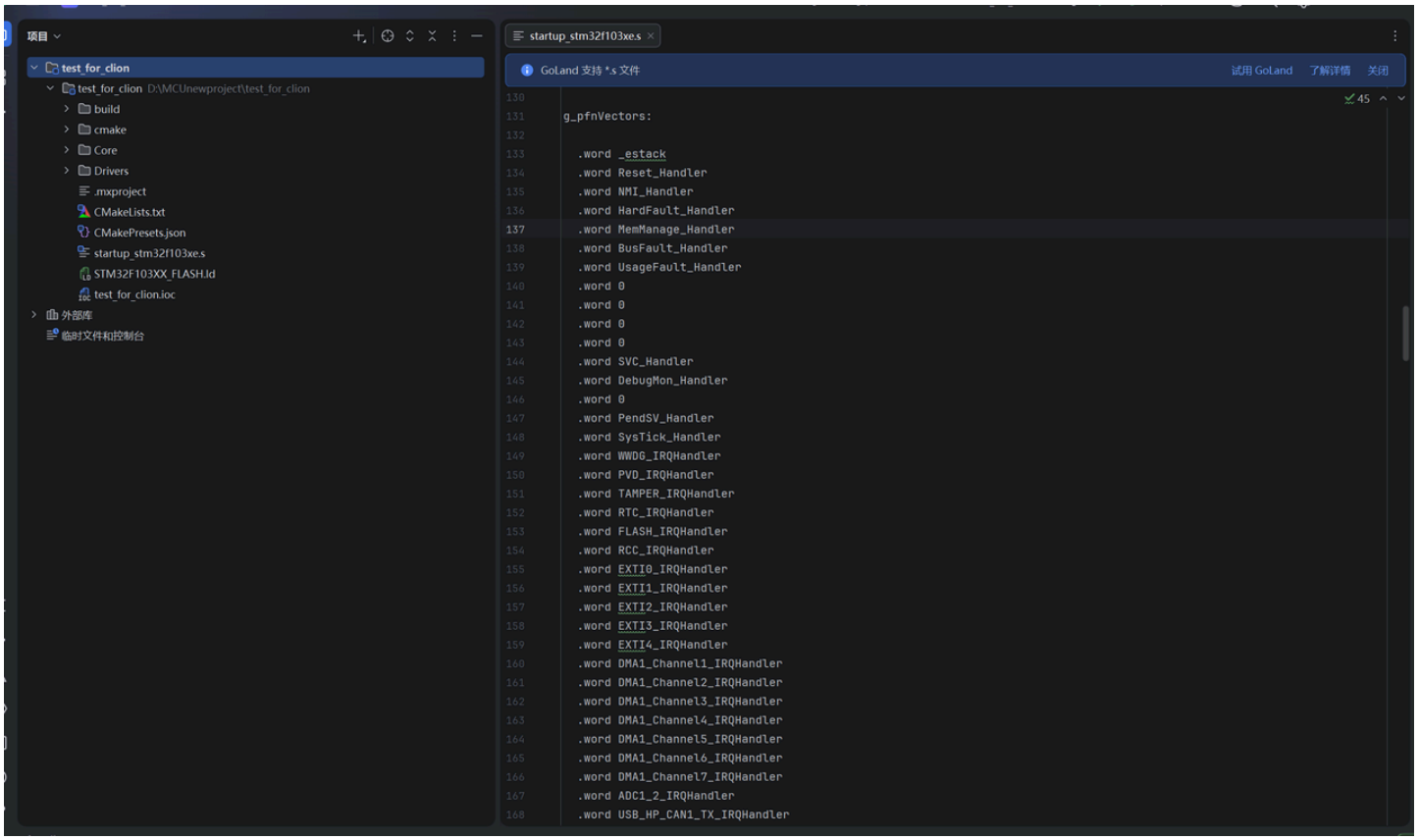
2.7 中断向量表

中断向量表是单片机处理中断机制的核心数据结构，其作用类似于硬件与中断服务程序之间的"导航地图"。它直接决定了系统在中断触发时如何快速定位到对应的处理代码。

中断向量表存放着中断服务函数的首地址，由四字节对齐，由地址找到我们对应的中断服务函数。中断向量表定义在startup启动文件中，当发生中断时，CPU会根据中断向量表找到要执行的中断服务函

数的首地址，自动执行对应的中断服务函数。

接下来我们打开Clion,来看一下我们中断向量表。



如图所示，打开我们项目的startup_stm32f103xe.s文件，找到g_pfnVectors：往下的部分即可看到中断向量表。

三、EXTI——外部中断

3.1 初识外部中断

外部中断是指微控制器（如STM32）中一种由**外部硬件信号触发**的中断机制，用于快速响应引脚（通常是我们刚刚学过的GPIO引脚）电平变化（如按键按下、传感器信号）触发的中断。

注意：外部中断的"外部"可不是单片机外部哦，而是内核外部，也就是上文所说的片上。在后续的学习中我们还会接触很多不同种的中断，本次我们先来学习GPIO电平跳变而引起的外部中断。（有能力的同学可以多了解一些其他中断，如定时中断。）

3.2 EXTI是什么?

EXTI（外部中断/事件控制器）作为STM32的一个外设，是STM32中用于处理外部中断和事件的模块。EXTI可分为两大部分功能：

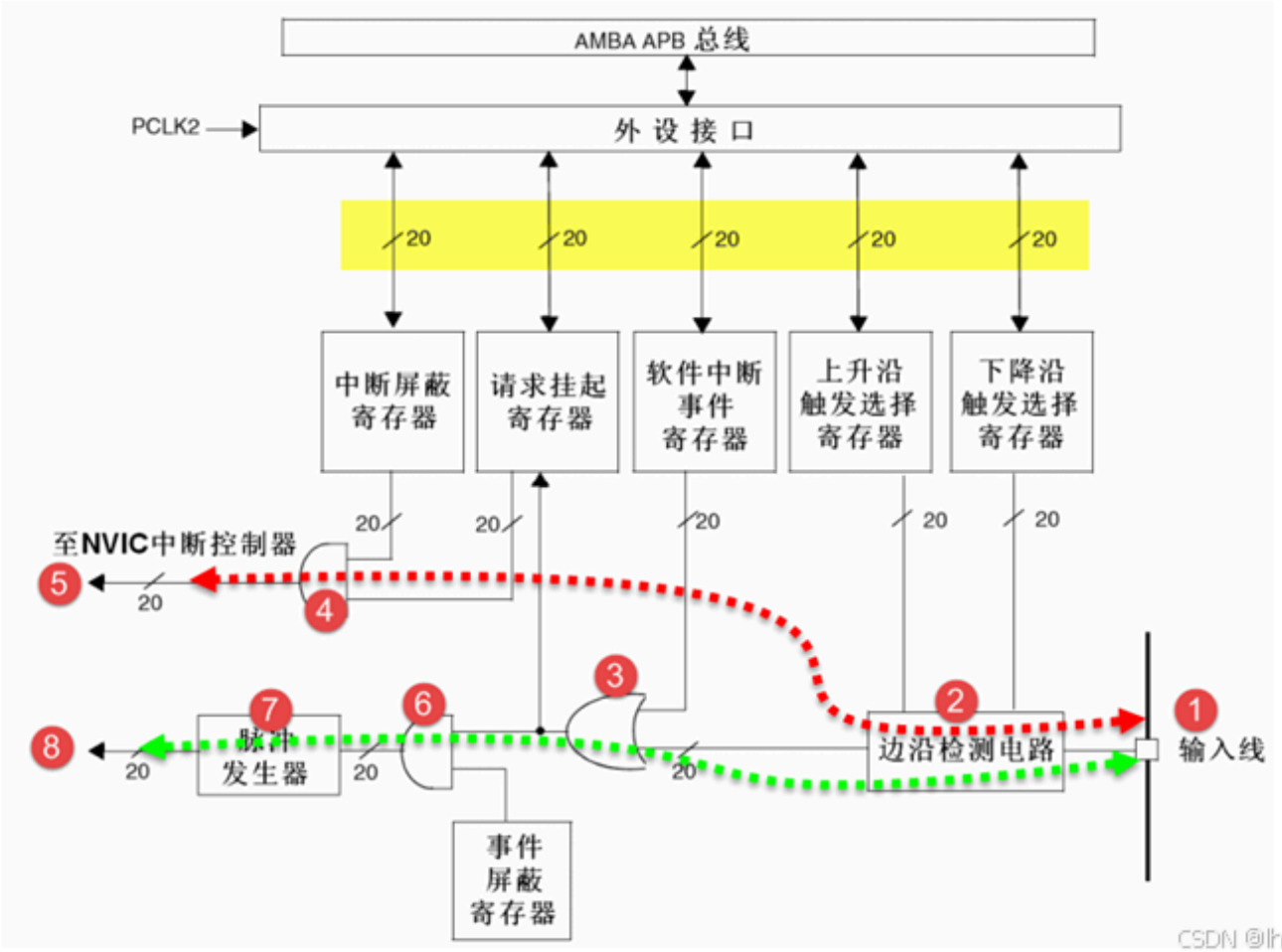
- **产生中断**：当检测到指定GPIO引脚上的电平变化时，产生一个中断请求，最终信号流入NVIC控制器内。

- **产生事件**：当检测到指定GPIO引脚上的电平变化时，产生一个事件信号，可用于软件轮询或其他外设（如DMA）。

这两个功能从硬件上就有所不同。第二个功能同学们可以自行了解，后续会讲。

3.3 外部中断触发完整流程

首先我们来看EXTI功能框图中红色虚线指示的电路流程，它是一个产生中断的线路，最终信号流入到NVIC控制器内。



编号1：输入线

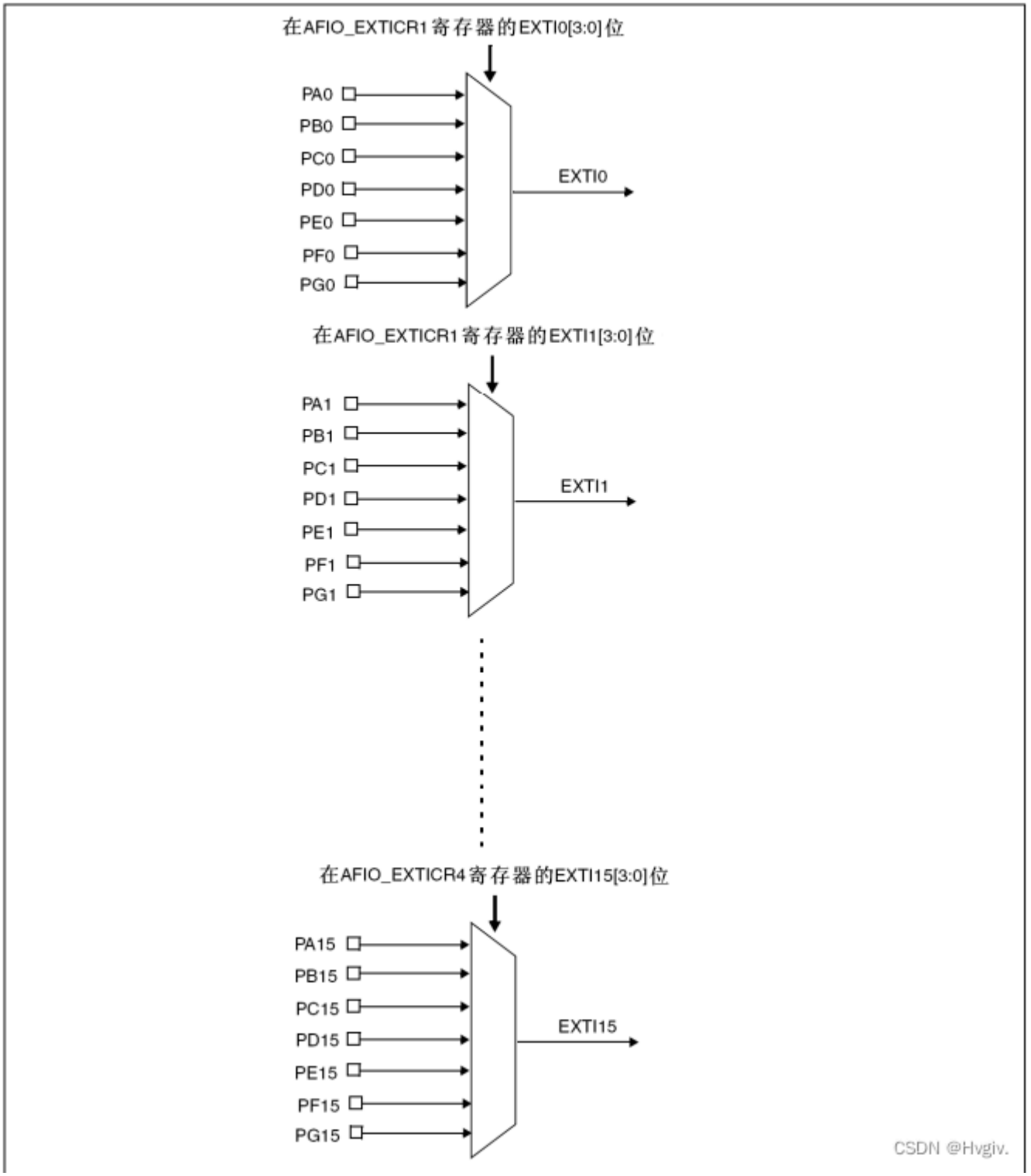
EXTI控制器有19个中断/事件输入线，这些输入线可以通过寄存器设置为任意一个GPIO，也可以是一些外设的事件。

在STM32中，每一个GPIO都可以作为外部中断的触发源，外部中断一共有16根线，但STM32的GPIO口多达上百个，同时每个GPIO都有很多引脚，如果每个GPIO的引脚都占用一个EXTI中的GPIO通道，那么EXTI中的GPIO通道明显不够用，那么怎么办？

单片机通过**AFIO**实现中断引脚选择。什么是AFIO？AFIO实际上就是一个数据选择器，它可以选择所有GPIO的其中的16个引脚接入到EXTI的通道中。这样，众多GPIO口就被成功分组，一条外部中断线对应相应的一组GPIO口。112个通用I/O端口以下图的方式连接到16个外部中断/事件线上。

如图：

图20　外部中断通用I/O映像



每一条外部中断都可以与任意一组的对应引脚相连，但不能重复使用。比如，外部中断Line0可以和PA0、PB0、PC0等任意一条0号引脚相连，但如果已经和PA0相连接，则不能同时与PB0、PC0等0号引脚相连。

对数字比较敏感的同学可能会疑问，为什么介绍编号1的时候说EXTI有19条中断/事件输入线，而112个GPIO口最终只连接到了16条中断线上呢？那是因为不仅是GPIO可以作为外部中断的输入，还有其他的：

- EXTI_Line16为PVD输出（EXTI的电源电压检测中断）

- EXTI_Line17为RTC Alarm（RTC闹钟唤醒中断）
- EXTI_Line18为USB唤醒事件

这三条中断线我们不会用到，只做了解即可。

外部中断的触发方式

外部中断的触发主要通过GPIO口的电平变化来触发。外部中断支持GPIO三种电平跳变模式：

1. **上升沿中断**：当GPIO的电平从低电平跳变成高电平时，引发外部中断。
2. **下降沿中断**：当GPIO的电平从高电平跳变成低电平时，引发外部中断。
3. **上升沿和下降沿中断**：当GPIO的电平从低电平跳变成高电平时，或者从高电平跳变成低电平时，都可以引发外部中断。

编号2：边沿检测电路

边沿检测电路会根据上升沿触发选择寄存器（EXTI_RTSR）和下降沿触发选择寄存器（EXTI_FTSR）对应位的设置来控制信号触发。边沿检测电路以输入线作为信号输入端，如果检测到有边沿跳变就输出有效信号1给编号3电路，否则输出无效信号0。

而EXTI_RTSR和EXTI_FTSR两个寄存器可以控制需要检测哪些类型的电平跳变过程，可以是只有上升沿触发、只有下降沿触发，或者上升沿和下降沿都触发。

流程总结

总体来说，当GPIO口发生电平跳变时，通过AFIO选择对应的中断引脚，通向EXTI边沿检测，最后通过中断通道至NVIC中断控制器发出中断。

> 同学们也可点开链接查看更详细的视频讲解：B站EXTI视频讲解

3.4 EXTI相关寄存器

前面讲了EXTI的硬件流程，这里把EXTI用到的核心寄存器整理一下，方便大家对照理解。

寄存器	全称	作用	关键位
EXTI_IMR	中断屏蔽寄存器	某位写1开启对应线的中断请求，写0屏蔽	MRx（x=0~19）
EXTI_RTSR	上升沿触发选择寄存器	某位写1使能对应线的上升沿触发	TRx（x=0~19）
EXTI_FTSR	下降沿触发选择寄存器	某位写1使能对应线的下降沿触发	TRx（x=0~19）
EXTI_PR	挂起寄存器	中断发生后对应位变1，写1清除该位	PIFx（x=0~19）

这四个寄存器正好对应外部中断的完整操作流程：

1. **EXTI_IMR**——使能对应线的中断（开关）
2. **EXTI_RTSR / EXTI_FTSR**——选择触发方式（上升沿/下降沿/双边沿）
3. **EXTI_PR**——中断发生后置位，处理完后写1清除（不清会反复进中断）

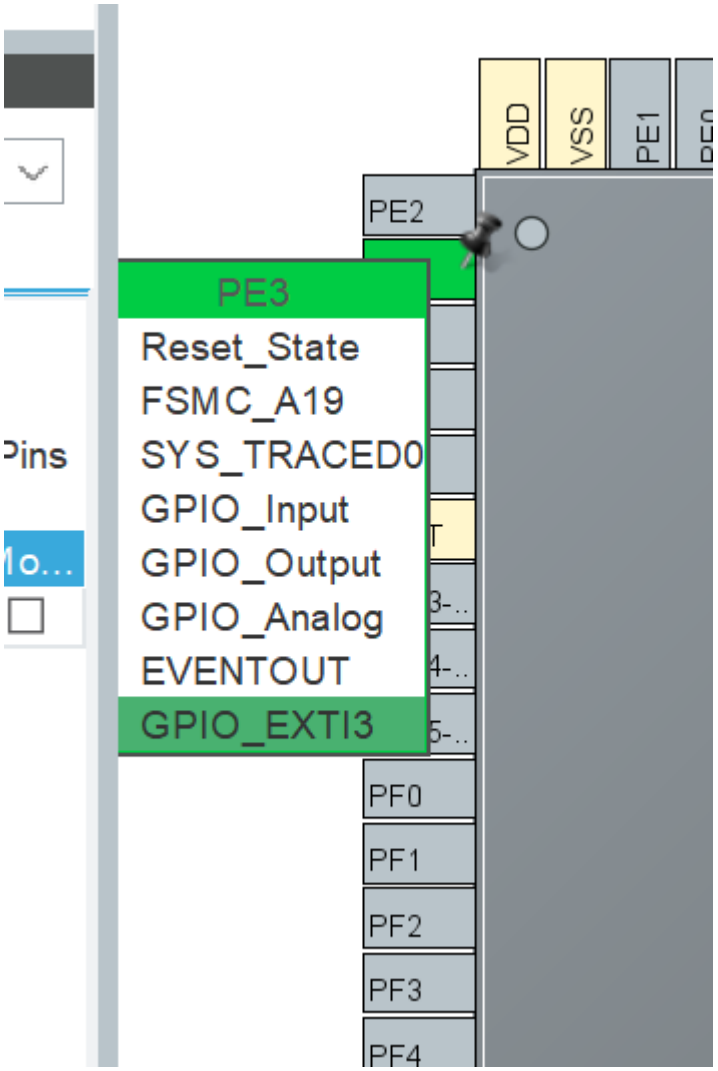
> 注意：EXTI_PR的清除方式是**写1清除**，不是写0。这是很多初学者容易踩的坑。不过在HAL库中，`HAL_GPIO_EXTI_IRQHandler` 已经帮你做好了清除标志位的工作，你不需要手动操作这个寄存器。

3.5 外部中断在CubeMX中的配置

我们这里以PE3引脚作为举例。

第一步：配置GPIO引脚

将PE3号引脚设置为按键的输入引脚，将其设置为外部中断模式。



接着点开GPIO标签页，对引脚进行如下设置：

- 将GPIO模式设置为**上升/下降沿触发的外部中断**
- 上下拉电阻：按键的上下拉电阻配置需根据硬件电路设计决定。我们可以打开单片机的原理图，原理图上面说明了是上/下拉电阻，则后续可以不配置；若未说明，则证明是悬空，必须自行配置。我们一般都配置为**上拉电阻**。
- 设置用户标签（User Label）：给这个引脚起名字，方便记忆。

如图所示：



CubeMX配置与寄存器的对应关系：

- GPIO模式选"External Interrupt Mode with Rising/Falling edge trigger" → 本质是通过AFIO将PC1映射到EXTI_Line1，并配置 `EXTI_RTSR` 和 `EXTI_FTSR` 的对应位
- 上下拉电阻配置 → 配置GPIO的 `PUPDR` 寄存器（这是GPIO的寄存器，不是EXTI的）

第二步：配置NVIC

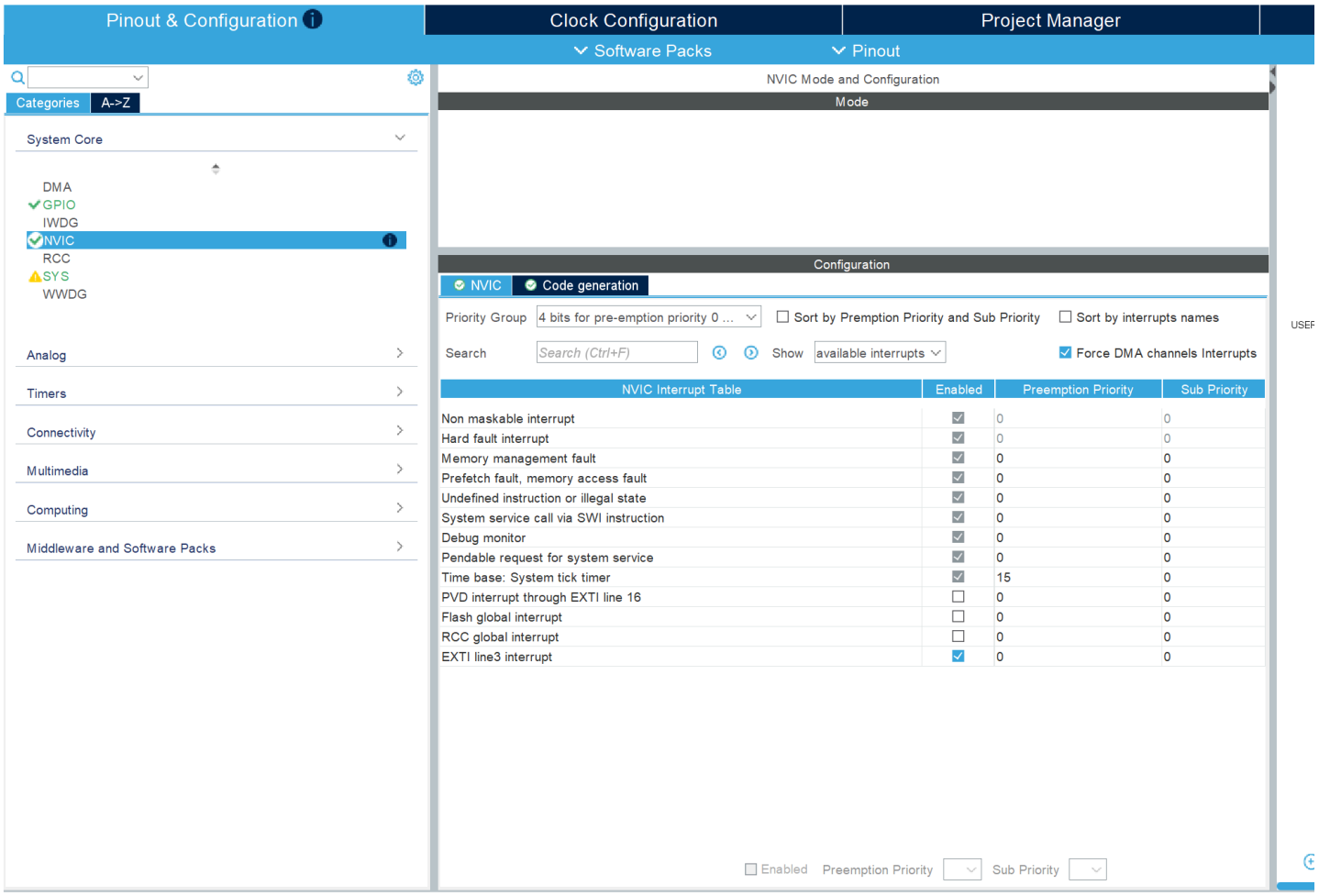
在NVIC标签页下，开启对应的外部中断。

在CubeMX中，通过NVIC标签页：

- **NVIC Interrupt Table**是中断的名称

- **Enabled**是中断是否开启的标志，打勾即为开启

将EXTI线Line1的中断设置为"Enabled"，表示启用了EXTI线1中断功能。后面的抢占优先级（Preemption Priority）和响应优先级（Sub Priority）的级别我们可以自行更改，这里我们不做更改，保证都为最高级别0，避免有其他中断干扰我们的操作。



> **CubeMX配置与寄存器的对应关系：**
>
> - Enabled勾选 → 写 `NVIC_ISER` 寄存器的对应位，使能中断
> - Preemption Priority / Sub Priority设置 → 写 `NVIC_IPR` 寄存器的高4位，具体怎么分由 `AIRCR` 的优先级分组决定

这样我们就成功的将PC1的外部中断开启了。

3.6 HAL库函数讲解

生成代码后，首先我们打开 `gpio.c` 文件，找到EXTI interrupt init这一部分。CubeMX生成的代码将我们配置的NVIC初始化。

HAL_NVIC_SetPriority

代码块

```
1   void HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t
    SubPriority);
```

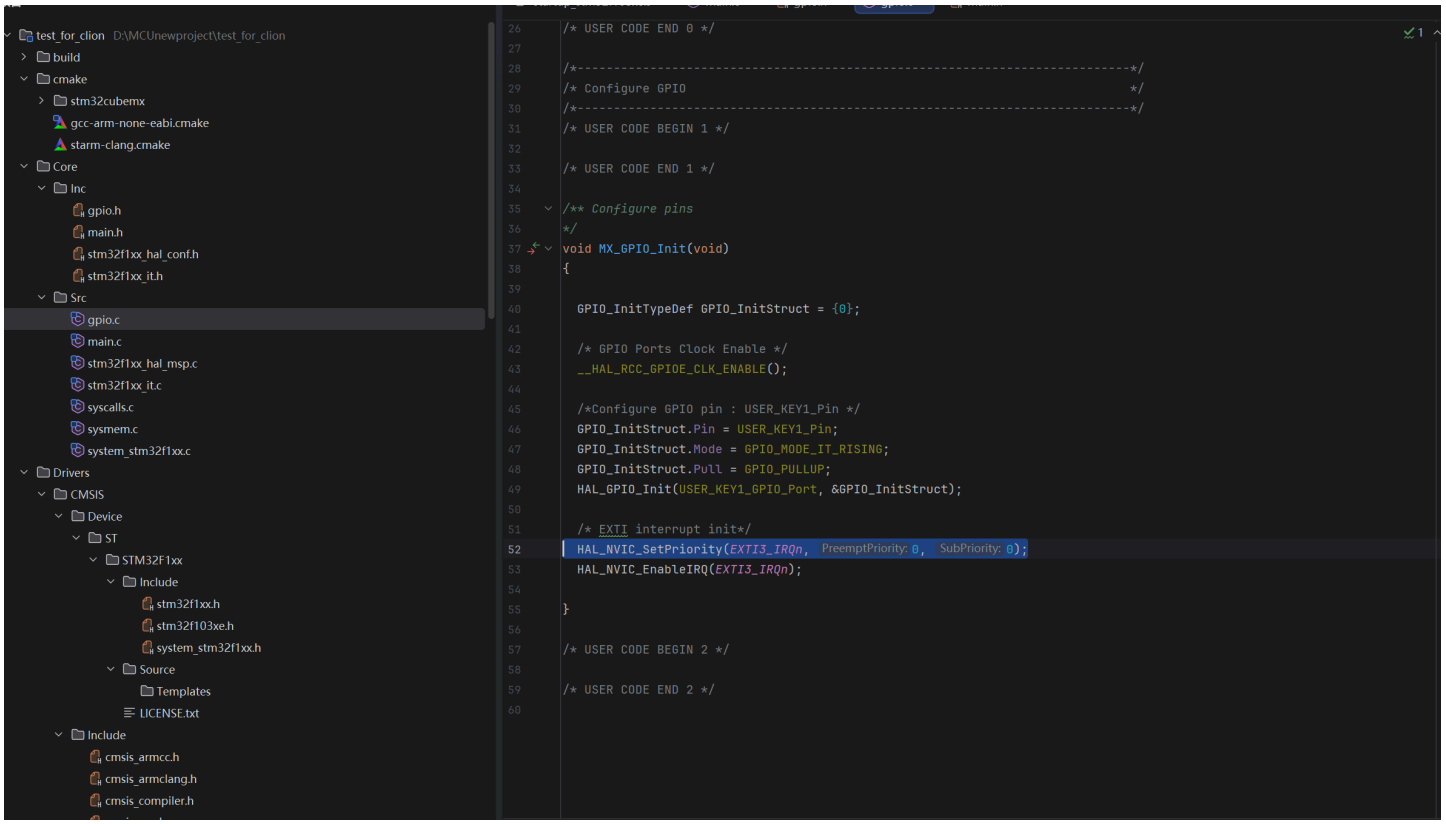
- **功能**：设置指定中断的抢占优先级和响应优先级

- **参数**：

- `IRQn`：中断编号，如 `EXTI1_IRQn`
- `PreemptPriority`：抢占优先级
- `SubPriority`：响应优先级

- **返回值**：无

使用 `HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);` 设置优先级。该优先级的设置与我们在CubeMX中配置的一样，第一个0对应的是抢占优先级（Preemption Priority）的等级，第二个0对应的是响应优先级（Sub Priority）的等级。当我们在CubeMX中修改相应的等级时，该代码也会相应的修改。



HAL_NVIC_EnableIRQ

代码块

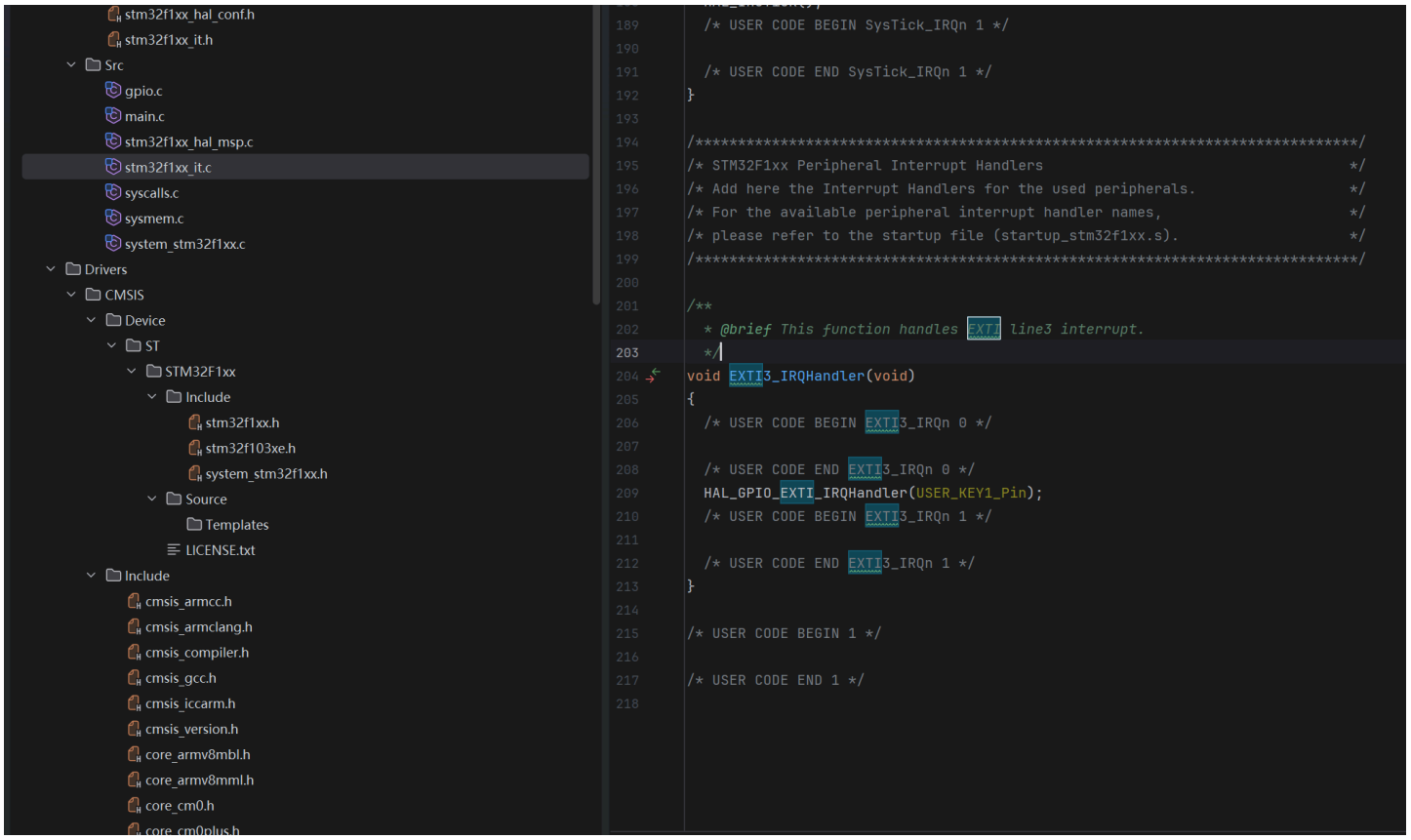
```
1   void HAL_NVIC_EnableIRQ(IRQn_Type IRQn);
```

- **功能**：使能指定中断（写NVIC_ISER寄存器）
- **参数**：`IRQn`——中断编号，如`EXTI1_IRQn`
- **返回值**：无

使用 `HAL_NVIC_EnableIRQ(EXTI1_IRQn);` 使能中断。

EXTI_IRQHandler——中断服务函数

当我们打开EXTI Line3 interrupt时，并且在CubeMX的Code generation中配置了generate IRQ handler，CubeMX会自动为我们生成此函数。我们可以打开 `stm32f1xx_it.c` 文件看到此函数。



同理，如果打开EXTI Line2 interrupt，会生成`EXTI2_IRQHandler`；打开EXTI Line1 interrupt，生成`EXTI1_IRQHandler`，以此类推。

它就是我们触发外部中断（比如按下按键触发中断后）STM32会调用执行的中断处理函数。中断向量表中存放了该函数的首地址，中断发生后，CPU根据中断向量表找到该函数的首地址，接着CPU根据首地址来到该函数并开始运行。

HAL_GPIO_EXTI_IRQHandler与HAL_GPIO_EXTI_Callback

进入中断服务函数后，程序将再进入`HAL_GPIO_EXTI_IRQHandler`函数。此函数包含两个部分：

1. 清除特定GPIO引脚的外部中断挂起标志位的底层宏，确保中断处理完成后不再触发重复中断
2. 调用`HAL_GPIO_EXTI_Callback`回调函数

代码块

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);
```

- **功能**：外部中断回调函数，弱函数，用户重写后在中断触发时自动调用

- **参数**：`GPIO_Pin`——触发中断的引脚号，如`GPIO_PIN_1`
- **返回值**：无
  HAL库中实现：./Drivers/STM32F1xx_HAL_Driver/Src/stm32f1xx_hal_gpio.c

```c
/**
  * @brief  This function handles EXTI interrupt request.
  * @param  GPIO_Pin: Specifies the pins connected EXTI line
  * @retval None
  */
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}

/**
  * @brief  EXTI line detection callbacks.
  * @param  GPIO_Pin: Specifies the pins connected EXTI line
  * @retval None
  */
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(GPIO_Pin);
  /* NOTE: This function Should not be modified, when the callback is needed,
           the HAL_GPIO_EXTI_Callback could be implemented in the user file
   */
}
```

`HAL_GPIO_EXTI_Callback`是一个弱函数，当外部中断被触发时，系统会自动调用该函数，并将触发中断的GPIO的引脚号作为参数传入。如果想要应用此函数，只需要不加weak在主程序中写一个同名函数即可，将自己的函数写入该同名函数中。

> 还记得我们第一章讲的回调函数和弱函数吗？`HAL_GPIO_EXTI_Callback`就是它们的典型应用——HAL库在`EXTI_IRQHandler`里调用了这个弱函数，你写一个同名函数覆盖它，中断发生时就会自动执行你的逻辑。

对比总结

	EXTI_IRQHandler	HAL_GPIO_EXTI_Callback
性质	中断服务程序	中断回调函数
调用者	由CPU硬件自动调用	由HAL库在EXTI_IRQHandler内部调用
主要职责	清除中断标志位，管理中断流程	执行用户的应用逻辑代码
代码位置	通常已由HAL库实现，不应直接修改	需要由用户在工程中重写
安全性	必须小心处理标志位，否则会卡死	更安全，标志位已由HAL库清除
可移植性	与具体芯片和HAL库实现强相关	应用逻辑与底层硬件隔离，可移植性更好

所以，我们想要在中断中写自己的代码时，不要去修改或重写 `EXTI0_IRQHandler`、`EXTI1_IRQHandler` 等函数。相信HAL库已经为你做好了底层工作。要在你自己的代码文件中，重写 `HAL_GPIO_EXTI_Callback` 函数，将你所有响应中断的应用程序代码都写在 `HAL_GPIO_EXTI_Callback` 函数内部。

这种"中断服务程序 + 回调函数"的架构是HAL库的核心设计思想之一，它很好地分离了底层硬件操作和上层应用逻辑，使得代码更清晰、更安全、更易于维护和移植。

3.7 注意：中断里不能用HAL_Delay

`HAL_Delay()` 是一个延时函数，输入我们想要延长的时间，单位默认是毫秒，程序运行到这里的时候便会在这里停留你设置的那段时间，之后再继续运行下面的程序。

但是——**千万不要在中断里用延时函数！**

当我们在中断中使用 `HAL_Delay()` 函数时，当中断触发条件发生时，跳入到存在 `HAL_Delay()` 函数的延时中断中，会出现死机的情况。这是因为中断优先级的缘故：中断优先级高的能抢占CPU的资源，延时函数中含有一个优先级较低的中断用来记录时间，由于中断优先级比较低，而当前正在处于优先级比较高的中断，所以无法进入到用来计时的中断，那么延时函数中的计时就会停止。

`HAL_Delay()` 是通过轮询系统时间阻塞住CPU直到到了时间才会跳出循环的延时函数，而计时中止了， `HAL_Delay()` 也无法到达设定值，所以程序将一直卡在这。

> 这里是比较笼统的说法，如果了解定时器等知识的同学可以点开链接更详细地看一看具体原因：
> CSDN博客——为什么中断里不能用HAL_Delay

3.8 按键消抖

如果按键感觉不灵敏，小同学们可以了解一下按键消抖。

当机械触点断开、闭合时，即按下按钮和松开按钮时，由于机械触点的弹性作用，一个按键开关在闭合时不会马上稳定地接通，在断开时也不会一下子断开。因而在闭合及断开的瞬间均伴随有一连串的抖动。为了不产生这种现象而作的措施就是按键消抖。

单片机如果在触点抖动期间检测按键的通断状态，则可能导致判断出错，即按键一次按下或释放被错误地认为是多次操作，从而引起误处理。因此，为了确保单片机对一次按键动作只作一次响应，就必须考虑如何消除按键抖动的影响。

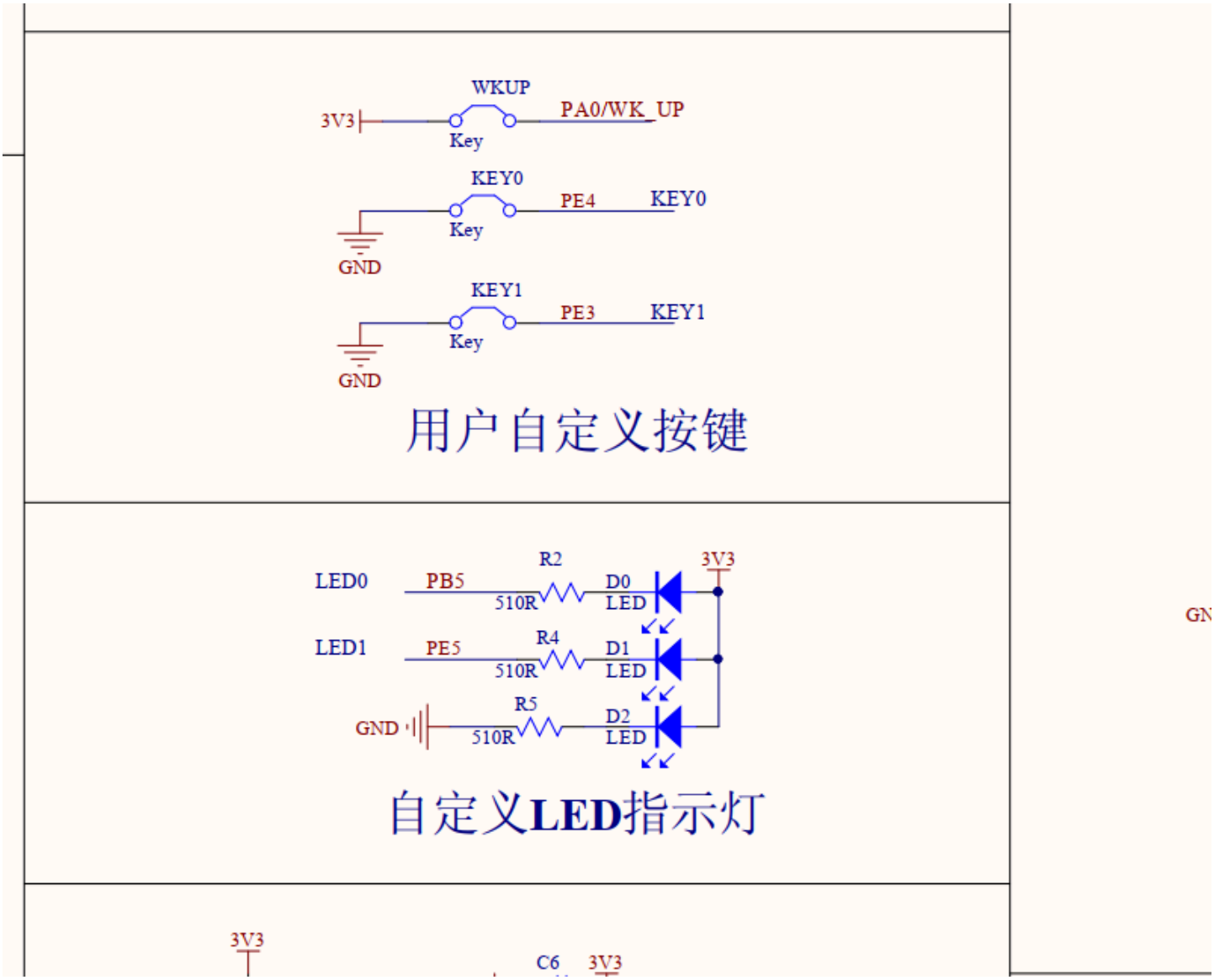
按键消抖有很多方式，我们主要是要软件消抖，从我们的代码层面来消抖。最常用的方法是：检测出键闭合后执行一个延时程序，5ms~10ms的延时，让前沿抖动消失后再一次检测键的状态，如果仍保持闭合状态电平，则确认为真正有键按下。当检测到按键释放后，也要给5ms~10ms的延时，待后沿抖动消失后才能转入该键的处理程序。

同学们可以根据这段描述编写出代码用于按键消抖。

四、项目实战

任务1：单按键控制LED亮灭（toggle）

要求：利用一个按键控制小灯的亮灭，即按下一个按键，小灯亮起，再按下相同的按键，小灯熄灭。



伪代码参考方案：

代码块

```
HAL_GPIO_EXTI_Callback(GPIO_Pin){
    if (GPIO_Pin == KEY_PIN){
        //反转小灯引脚
    }
}

```

任务2：双按键控制LED不同频率闪烁

要求：利用两个按键控制一个小灯，按一个按键小灯高频率闪烁，按另一个按键小灯低频率闪烁。

> 小tip：中断里不能用延时函数，所以不要把控制小灯的代码放在中断回调函数里，想办法把小灯闪烁的代码放在while循环里吧。

伪代码参考方案：

代码块

```
uint32_t toggle_time = 0;

HAL_GPIO_EXTI_Callback(GPIO_Pin):
    if GPIO_Pin == KEY1_PIN:
        toggle_time = ;//你想要停止的时间
    if GPIO_Pin == KEY2_PIN:
        toggle_time = ;//你想要停止的时间

while(1):
    //反转小灯引脚
    //延长 toggle_time ms
```

> 注：方式不唯一，鼓励使用其他逻辑去写

任务3：打断与被打断

要求：

- 两个按键：KEY1 配置**低**抢占优先级（比如 2），KEY2 配置**高**抢占优先级（比如 0）
- KEY1 的中断回调里放一个**较长的 for 循环空转**（比如 5000 万次空循环，模拟一个耗时操作），循环结束后点亮 LED1
- KEY2 的中断回调里直接点亮 LED2，不做任何延时
- **操作**：先按下 KEY1 触发低优先级中断，在它还在空转的时候（LED1 还没亮），立刻按下 KEY2 触发高优先级中断
- **观察**：LED2 是否在 LED1 之前亮？（高优先级打断了低优先级）
- **反向测试**：交换优先级 ——KEY1 高、KEY2 低，先按 KEY1 再按 KEY2，观察 LED2 是否要等 LED1 亮了之后才亮

伪代码思路：

代码块

```
// KEY1中断回调（低优先级）
```

```
HAL_GPIO_EXTI_Callback(GPIO_Pin):
    if GPIO_Pin == KEY1_PIN:
        // 模拟耗时操作（仅用于演示嵌套！实际开发不要这么写）
        for(uint32_t i = 0; i < 50000000; i++) {
            __NOP();  // 空操作，占CPU但不做实事
        }
         // 空转结束才亮

// KEY2中断回调（高优先级）
HAL_GPIO_EXTI_Callback(GPIO_Pin):
    if GPIO_Pin == KEY2_PIN:
       // 立刻亮
```

注意事项:

- for 循环空转只是为了演示本题的嵌套效果，实际开发中**绝对不要在中断里写耗时循环**。
- 空转次数根据芯片主频调整，F103 主频 72MHz，5000 万次大约 0.5~1 秒，如果需要更长时间请适当增大
- `__NOP()` 是空操作指令，比空的 for 循环更精确（编译器不会优化掉）

五、思考题

1.假设两个中断A和B,抢占优先级相同，响应优先级A>B,那么当A,B同时触发时，谁先执行？当B正在执行时，A触发了，A能否打断B?当A正在执行时，B触发了，B能否打断A?

2.请思考一下中断向量表的作用，以及在 STM32 中它是怎么工作的?

3.完整思考一下外部中断的整个工作流程（从GPIO口电平跳变一直到NVIC发挥作用），既要思考EXTI的工作流程，也要思考NVIC的工作过程。

4.思考一下抢占优先级与响应优先级之间的关系。

——By sjy

QQ:3618596746